JAVA DSA: GREEDY ALGORITHMS

Complete interview-focused guide • Concepts • Greedy rules • Proof intuition • Java implementations • Dry runs

What this PDF contains

• Greedy fundamentals: how to recognize a greedy problem, how to justify a local choice, and when greedy fails.

• A broad set of standard greedy algorithms and high-frequency interview implementation problems.

• Runnable Java method implementations, example inputs/outputs, time and space complexity, and edge-case warnings.

• Graph greedy algorithms: Kruskal, Prim, and Dijkstra; scheduling, intervals, heaps, strings, arrays, and resource allocation.

Scope note: No finite guide can contain every greedy algorithm ever invented. This guide covers the major standard algorithms and interview patterns. Some problems that look greedy actually require Dynamic Programming; those are clearly identified.

1. Greedy algorithm: foundation

Definition: A greedy algorithm constructs a solution step by step. At each step it chooses the option that looks best according to a local rule, commits to that choice, and does not normally reconsider it.

Simple meaning: Har step par abhi ka sabse useful choice lo — lekin choice aisi honi chahiye jo final optimal answer ko possible banaye.

Greedy workflow

• Define the objective: maximize value, minimize cost/time/count, or build a valid structure.

• List the choices available at each step.

• Write an exact selection rule (earliest finish, smallest edge, largest gain, etc.).

• Check feasibility before accepting a choice.

• Update the state and repeat until finished.

• Prove why a locally best choice can be part of some globally optimal solution.

Two properties to understand

Greedy-choice property: There exists an optimal solution that begins with the greedy choice. **Optimal substructure:** After making that choice, the remaining subproblem can also be solved optimally.

A common proof is an **exchange argument**: take an optimal solution, replace its first choice with the greedy choice, and show that the solution remains feasible and no worse.

How to recognize common patterns

Signal in question	Likely greedy strategy
Maximum number of non-overlapping intervals	Sort by earliest finishing time
Minimum resources for overlapping jobs	Sort events + min-heap of end times
Minimum cost to connect all vertices	MST: Kruskal / Prim
Shortest path with non-negative edges	Dijkstra
Repeatedly combine two smallest items	Min-heap: Huffman / optimal merge
Choose best available profit under constraints	Sort + priority queue
Remove digits to make smallest number	Monotonic increasing stack
Cover a line with minimum intervals	Sort by start; extend farthest reach

When greedy is NOT automatically correct

A locally attractive choice may block a better global answer. Example: coin denominations {1, 3, 4}, amount 6. Greedy takes 4 + 1 + 1 (3 coins), while the optimal answer is 3 + 3 (2 coins). For arbitrary denominations, coin change needs DP or a proof specific to the denomination system.

Also distinguish **fractional knapsack** (greedy by value/weight is optimal) from **0/1 knapsack** (that same greedy rule is not generally optimal; use DP or other methods).

2. Interval, scheduling and coverage algorithms

2.1 Activity selection / maximum meetings

Core idea: Choose the largest number of mutually non-overlapping activities.

Greedy choice / rule: Sort by increasing finish time. Select the first activity, then select the next activity whose start is at least the finish of the last selected one.

Complexity: Time O(n log n) for sorting; O(n) scan. Extra space depends on sorting.

Java implementation

```
import java.util.*;

class ActivitySelection {
    static class Activity {
        int start, end;
        Activity(int s, int e) { start = s; end = e; }
    }

    static List<Activity> select(Activity[] a) {
        Arrays.sort(a, Comparator.comparingInt(x -> x.end));
        List<Activity> chosen = new ArrayList<>();
        int lastEnd = Integer.MIN_VALUE;

        for (Activity x : a) {
            if (x.start >= lastEnd) {
                chosen.add(x);
                lastEnd = x.end;
            }
        }
        return chosen;
    }
}
```

Dry run / example: Activities (1,3), (2,5), (4,7), (6,9). Sort by finish: choose (1,3), then (4,7). Answer count = 2 (one valid optimal selection).

Important edge cases: Clarify whether touching endpoints are allowed. If one activity ending at 3 and another starting at 3 may coexist, use start >= lastEnd; otherwise use start > lastEnd.

2.2 Minimum meeting rooms / minimum platforms

Core idea: Find the minimum number of rooms/platforms needed so overlapping intervals can all be served.

Greedy choice / rule: Sort start times and end times separately. If the next start is before the earliest end, allocate another room; otherwise reuse a room.

Complexity: Time O(n log n); space O(n) for copied arrays.

Java implementation

```
import java.util.*;

class MeetingRooms {
    static int minRooms(int[][] meetings) {
        int n = meetings.length;
        if (n == 0) return 0;
        int[] start = new int[n], end = new int[n];
        for (int i = 0; i < n; i++) {
            start[i] = meetings[i][0];
            end[i] = meetings[i][1];
        }
        Arrays.sort(start);
        Arrays.sort(end);

        int rooms = 0, maxRooms = 0, i = 0, j = 0;
        while (i < n) {
            if (start[i] < end[j]) {
                rooms++;
                maxRooms = Math.max(maxRooms, rooms);
                i++;
            } else {
                rooms--;
                j++;
            }
        }
        return maxRooms;
    }
}
```

Dry run / example: Meetings [0,30], [5,10], [15,20]. At time 5 a second room is needed; at 10 one frees; at 15 reuse it. Maximum rooms = 2.

Important edge cases: If meetings touching at the same timestamp can reuse a room, use start < end as above. If the problem defines inclusive endpoints, adjust tie handling.

2.3 Non-overlapping intervals / erase minimum intervals

Core idea: Remove the fewest intervals so that the remaining intervals do not overlap.

Greedy choice / rule: Equivalent to keeping the maximum compatible set: sort by end time, greedily keep each interval that starts at or after the previous kept end.

Complexity: Time O(n log n); space O(1) beyond sorting.

Java implementation

```
import java.util.*;

class EraseOverlap {
    static int eraseOverlapIntervals(int[][] a) {
        if (a.length == 0) return 0;
        Arrays.sort(a, Comparator.comparingInt(x -> x[1]));
        int kept = 0, lastEnd = Integer.MIN_VALUE;

        for (int[] in : a) {
            if (in[0] >= lastEnd) {
                kept++;
                lastEnd = in[1];
            }
        }
        return a.length - kept;
    }
}
```

Dry run / example: Intervals [1,2], [2,3], [1,3]. Keep [1,2] and [2,3]; remove [1,3]. Minimum removals = 1.

Important edge cases: Check endpoint convention and empty input.

2.4 Minimum arrows to burst balloons

Core idea: Find the fewest points that intersect every interval.

Greedy choice / rule: Sort by interval end. Place an arrow at the end of the first uncovered interval; skip every interval containing that point.

Complexity: Time O(n log n); space O(1) beyond sorting.

Java implementation

```
import java.util.*;

class MinArrows {
    static int findMinArrowShots(int[][] points) {
        if (points.length == 0) return 0;
        Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));
        int arrows = 1;
        long arrow = points[0][1];

        for (int[] p : points) {
            if (p[0] > arrow) {
                arrows++;
                arrow = p[1];
            }
        }
        return arrows;
    }
}
```

Dry run / example: Balloons [1,6], [2,8], [7,12], [10,16]. Arrow at 6 bursts first two; arrow at 12 bursts last two. Answer = 2.

Important edge cases: Use long for the arrow coordinate if inputs may approach integer extremes.

2.5 Minimum intervals to cover a target range

Core idea: Cover [0, target] using the fewest available intervals.

Greedy choice / rule: Sort by start. At current covered point, among all intervals starting no later than that point, choose the one extending farthest.

Complexity: Time O(n log n); scan O(n).

Java implementation

```
import java.util.*;

class MinIntervalsToCover {
    static int minCover(int[][] intervals, int target) {
        Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));
        int i = 0, covered = 0, used = 0, n = intervals.length;

        while (covered < target) {
            int farthest = covered;
            while (i < n && intervals[i][0] <= covered) {
                farthest = Math.max(farthest, intervals[i][1]);
                i++;
            }
            if (farthest == covered) return -1;
            covered = farthest;
            used++;
        }
        return used;
    }
}
```

Dry run / example: Target 10; intervals [0,4], [2,8], [7,10]. At 0 choose reach 4; at 4 choose reach 8; at 8 choose reach 10. Answer = 3.

Important edge cases: Works for a continuous line/range model. If target is not reachable, return -1. Be precise about open/closed endpoints.

2.6 Video stitching

Core idea: Minimum clips needed to cover time interval [0, T].

Greedy choice / rule: At each current time, inspect all clips that start at or before current time and choose the one with the farthest end.

Complexity: Time O(n log n); space O(1) beyond sorting.

Java implementation

```
import java.util.*;

class VideoStitching {
    static int videoStitching(int[][] clips, int T) {
        Arrays.sort(clips, Comparator.comparingInt(a -> a[0]));
        int i = 0, end = 0, farthest = 0, count = 0;

        while (end < T) {
            while (i < clips.length && clips[i][0] <= end) {
                farthest = Math.max(farthest, clips[i][1]);
                i++;
            }
            if (farthest == end) return -1;
            end = farthest;
            count++;
        }
        return count;
    }
}
```

Dry run / example: Clips [0,2], [1,5], [4,7], [6,10], T=10. Extend reach 0→5→10. Answer = 2.

Important edge cases: Do not advance current end unless a clip extends it; otherwise the process is stuck.

3. Knapsack, job sequencing and resource allocation

3.1 Fractional knapsack

Core idea: Maximize total value when an item can be taken fractionally.

Greedy choice / rule: Sort by value/weight descending; take each whole item while possible, then take the needed fraction of the next item.

Complexity: Time O(n log n); extra space O(1) beyond sorting.

Java implementation

```
import java.util.*;

class FractionalKnapsack {
    static class Item {
        double value, weight;
        Item(double v, double w) { value = v; weight = w; }
    }

    static double maxValue(Item[] items, double capacity) {
        Arrays.sort(items, (a, b) ->
            Double.compare(b.value / b.weight, a.value / a.weight));

        double total = 0.0;
        for (Item item : items) {
            if (capacity == 0) break;
            double take = Math.min(capacity, item.weight);
            total += take * (item.value / item.weight);
            capacity -= take;
        }
        return total;
    }
}
```

Dry run / example: Capacity 50; items (value,weight) = (60,10), (100,20), (120,30). Take first two fully (160), then 20/30 of third (80). Total = 240.

Important edge cases: Weight must be positive. This greedy proof does not apply to 0/1 knapsack.

3.2 Job sequencing with deadlines (unit-time jobs)

Core idea: Schedule at most one unit-duration job per time slot to maximize profit; each job earns profit only if completed by its deadline.

Greedy choice / rule: Sort jobs by descending profit. For each job, place it in the latest free slot not exceeding its deadline.

Complexity: Simple implementation: O(n log n + nD), where D is max deadline. With DSU predecessor optimization: near O(n log n + n α(D)).

Java implementation

```
import java.util.*;

class JobSequencing {
    static class Job {
        int id, deadline, profit;
        Job(int id, int d, int p) { this.id = id; deadline = d; profit = p; }
    }

    static int[] schedule(Job[] jobs) {
        Arrays.sort(jobs, (a, b) -> Integer.compare(b.profit, a.profit));
        int maxD = 0;
        for (Job j : jobs) maxD = Math.max(maxD, j.deadline);
        int[] slot = new int[maxD + 1];
        Arrays.fill(slot, -1);
        int count = 0, profit = 0;

        for (Job j : jobs) {
            for (int t = Math.min(maxD, j.deadline); t >= 1; t--) {
                if (slot[t] == -1) {
                    slot[t] = j.id;
                    count++;
                    profit += j.profit;
                    break;
                }
            }
        }
        return new int[]{count, profit};
    }
}
```

Dry run / example: Jobs (id,deadline,profit): (1,2,100), (2,1,19), (3,2,27), (4,1,25). Pick profit 100 in slot 2, then 27 in slot 1. Count=2, profit=127.

Important edge cases: Assumes every job takes exactly one time unit and deadlines are positive integer slots. For arbitrary durations, this simple greedy is not valid.

3.3 Assign cookies

Core idea: Maximize the number of children whose greed factor is satisfied by one cookie each.

Greedy choice / rule: Sort greed and cookie sizes. Give the smallest cookie that satisfies the least-greedy remaining child.

Complexity: Time O(n log n + m log m); O(1) extra apart from sorting.

Java implementation

```
import java.util.*;

class AssignCookies {
    static int findContentChildren(int[] greed, int[] cookies) {
        Arrays.sort(greed);
        Arrays.sort(cookies);
        int child = 0;
        for (int cookie : cookies) {
            if (child < greed.length && cookie >= greed[child]) child++;
        }
        return child;
    }
}
```

Dry run / example: Greed [1,2,3], cookies [1,1]. First cookie satisfies greed 1; second cannot satisfy greed 2. Answer = 1.

Important edge cases: Sorting order is essential; do not give a large cookie to a child who could have used a smaller one.

3.4 Maximum units on a truck

Core idea: Maximize units loaded with a fixed truck capacity and box types.

Greedy choice / rule: Sort box types by units per box descending; take as many as possible from the richest type.

Complexity: Time O(n log n); O(1) extra beyond sorting.

Java implementation

```
import java.util.*;

class MaximumUnits {
    static int maximumUnits(int[][] boxTypes, int truckSize) {
        Arrays.sort(boxTypes, (a, b) -> Integer.compare(b[1], a[1]));
        int units = 0;
        for (int[] type : boxTypes) {
            int take = Math.min(truckSize, type[0]);
            units += take * type[1];
            truckSize -= take;
            if (truckSize == 0) break;
        }
        return units;
    }
}
```

Dry run / example: Types [1,3], [2,2], [3,1], capacity 4. Take 1 box of 3 units and 2 boxes of 2 units and 1 box of 1 unit = 8.

Important edge cases: Check multiplication overflow; use long if counts and units are large.

3.5 IPO / maximize capital with at most k projects

Core idea: Choose at most k projects, each requiring capital and adding profit; maximize final capital.

Greedy choice / rule: Sort projects by required capital. As capital grows, add every affordable project to a max-heap by profit; choose the most profitable affordable project.

Complexity: Time O(n log n + k log n); space O(n).

Java implementation

```
import java.util.*;

class IPO {
    static int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
        int n = profits.length;
        int[][] projects = new int[n][2];
        for (int i = 0; i < n; i++) {
            projects[i][0] = capital[i];
```

```
            projects[i][1] = profits[i];
        }
        Arrays.sort(projects, Comparator.comparingInt(a -> a[0]));
        PriorityQueue<Integer> affordable = new PriorityQueue<>(Collections.reverseOrder());
        int i = 0;

        for (int step = 0; step < k; step++) {
            while (i < n && projects[i][0] <= w) {
                affordable.offer(projects[i][1]);
                i++;
            }
            if (affordable.isEmpty()) break;
            w += affordable.poll();
        }
        return w;
    }
}
```

Dry run / example: Initial capital 0; projects (capital,profit) (0,1),(1,2),(1,3), k=2. Choose profit 1, then among affordable choose profit 3. Final capital = 4.

Important edge cases: Standard formulation assumes selecting a project does not reduce capital; negative-profit projects should not be selected if stopping is allowed.

4. Greedy with heaps / optimal merging / coding

4.1 Huffman coding

Core idea: Build a prefix-free binary code with minimum weighted path length.

Greedy choice / rule: Repeatedly remove the two nodes with smallest frequency, combine them, and put the combined node back into a min-heap.

Complexity: Time O(n log n); space O(n).

Java implementation

```
import java.util.*;

class HuffmanCoding {
    static class Node {
        char ch;
        int freq;
        Node left, right;
        Node(char c, int f) { ch = c; freq = f; }
        Node(Node l, Node r) {
            ch = '#'; freq = l.freq + r.freq; left = l; right = r;
        }
    }

    static Node build(char[] chars, int[] freq) {
        PriorityQueue<Node> pq =
            new PriorityQueue<>(Comparator.comparingInt(x -> x.freq));
        for (int i = 0; i < chars.length; i++)
            pq.offer(new Node(chars[i], freq[i]));

        while (pq.size() > 1) {
            Node a = pq.poll(), b = pq.poll();
            pq.offer(new Node(a, b));
        }
        return pq.poll();
    }

    static void printCodes(Node root, String path) {
        if (root == null) return;
        if (root.left == null && root.right == null) {
            System.out.println(root.ch + " : " + (path.isEmpty() ? "0" : path));
            return;
        }
        printCodes(root.left, path + "0");
        printCodes(root.right, path + "1");
    }
}
```

Dry run / example: Frequencies A=5, B=9, C=12, D=13, E=16, F=45. Combine 5+9=14; 12+13=25; 14+16=30; 25+30=55; 45+55=100. Tree gives shorter codes to more frequent symbols.

Important edge cases: Different tie-breaking can produce different valid codes with the same optimal cost. A single-symbol alphabet needs a special code such as 0.

4.2 Connect ropes with minimum cost / optimal merge pattern

Core idea: Join all ropes into one while minimizing total cost; joining two ropes costs their combined length.

Greedy choice / rule: Always combine the two shortest ropes first using a min-heap.

Complexity: Time O(n log n); space O(n).

Java implementation

```
import java.util.*;

class ConnectRopes {
    static long minCost(long[] ropes) {
        PriorityQueue<Long> pq = new PriorityQueue<>();
        for (long x : ropes) pq.offer(x);
        long cost = 0;

        while (pq.size() > 1) {
            long a = pq.poll(), b = pq.poll();
            long sum = a + b;
            cost += sum;
            pq.offer(sum);
        }
        return cost;
    }
}
```

Dry run / example: Ropes [4,3,2,6]. Combine 2+3=5 (cost 5), 4+5=9 (cost 9), 6+9=15 (cost 15). Total cost = 29.

Important edge cases: Use long for sums. Empty or one-rope input has cost 0.

4.3 Minimum refuelling stops

Core idea: Reach a target with minimum refuelling stops; each station has fuel available and is passed in order.

Greedy choice / rule: Travel as far as current fuel allows. Whenever stuck, refuel at the largest-capacity station already passed (max-heap).

Complexity: Time O(n log n); space O(n).

Java implementation

```
import java.util.*;

class MinRefuelStops {
    static int minRefuelStops(int target, int startFuel, int[][] stations) {
        PriorityQueue<Integer> passed = new PriorityQueue<>(Collections.reverseOrder());
        int fuel = startFuel, i = 0, stops = 0;

        while (fuel < target) {
            while (i < stations.length && stations[i][0] <= fuel) {
                passed.offer(stations[i][1]);
                i++;
            }
            if (passed.isEmpty()) return -1;
            fuel += passed.poll();
            stops++;
        }
        return stops;
    }
}
```

Dry run / example: Target 100, start fuel 10; stations [10,60], [20,30], [30,30], [60,40]. At each point choose the largest fuel among passed stations; minimum stops = 2.

Important edge cases: Station positions must be sorted by distance. This standard version treats fuel units and distance units consistently.

5. Graph greedy algorithms

5.1 Kruskal minimum spanning tree (MST)

Core idea: Connect all vertices in an undirected weighted graph with minimum total edge weight and no cycle.

Greedy choice / rule: Sort edges by weight. Add an edge only when its endpoints are in different Disjoint Set Union (DSU) components.

Complexity: Time O(E log E) due to sorting; DSU operations are near O(1) amortized. Space O(V).

Java implementation

```
import java.util.*;

class KruskalMST {
    static class Edge {
        int u, v, w;
        Edge(int u, int v, int w) { this.u=u; this.v=v; this.w=w; }
    }
    static class DSU {
        int[] parent, size;
        DSU(int n) {
            parent = new int[n]; size = new int[n];
            for (int i=0;i<n;i++){ parent[i]=i; size[i]=1; }
        }
        int find(int x) {
            if (parent[x] != x) parent[x] = find(parent[x]);
            return parent[x];
        }
        boolean union(int a, int b) {
            a=find(a); b=find(b);
            if (a==b) return false;
            if (size[a] < size[b]) { int t=a; a=b; b=t; }
            parent[b]=a; size[a]+=size[b];
            return true;
        }
    }
    static long mstWeight(int n, Edge[] edges) {
        Arrays.sort(edges, Comparator.comparingInt(e -> e.w));
        DSU dsu = new DSU(n);
        long total=0; int used=0;
        for (Edge e : edges) {
            if (dsu.union(e.u,e.v)) {
                total += e.w;
                if (++used == n-1) break;
            }
        }
        return used == n-1 ? total : -1; // -1 means disconnected
    }
}
```

Dry run / example: Edges: 0-1(1), 1-2(2), 0-2(3), 2-3(4). Pick weights 1,2,4; skip 3 because it would form a cycle. MST weight = 7.

Important edge cases: Graph must be undirected. If fewer than V-1 edges are accepted, graph is disconnected and no spanning tree exists.

5.2 Prim minimum spanning tree (MST)

Core idea: Grow one tree until every vertex is included, minimizing the total weight.

Greedy choice / rule: Start from any vertex; repeatedly add the cheapest edge from the visited set to an unvisited vertex (min-heap).

Complexity: With adjacency list + heap: O((V+E) log V); space O(V+E).

Java implementation

```
import java.util.*;

class PrimMST {
    static class Pair {
        int to, weight;
        Pair(int t, int w) { to=t; weight=w; }
    }
    static long mst(List<List<Pair>> graph) {
        int n=graph.size();
        if (n==0) return 0;
        boolean[] seen=new boolean[n];
        PriorityQueue<Pair> pq=new PriorityQueue<>(Comparator.comparingInt(p->p.weight));
        pq.offer(new Pair(0,0));
        long total=0; int count=0;
```

```
        while (!pq.isEmpty()) {
            Pair cur=pq.poll();
            if (seen[cur.to]) continue;
            seen[cur.to]=true; total+=cur.weight; count++;
            for (Pair next: graph.get(cur.to))
                if (!seen[next.to]) pq.offer(next);
        }
        return count==n ? total : -1;
    }
}
```

Dry run / example: Start at vertex 0. If cheapest crossing edges are 0-1(1), 1-2(2), 2-3(4), MST total = 7.

Important edge cases: Undirected graph. The implementation returns -1 when not all vertices are reachable. For disconnected graphs, a minimum spanning forest is a different requested output.

5.3 Dijkstra shortest paths (non-negative edge weights)

Core idea: Find shortest distances from one source in a graph whose edge weights are all non-negative.

Greedy choice / rule: Repeatedly finalize the unvisited vertex with the smallest tentative distance; relax its outgoing edges.

Complexity: Adjacency list + priority queue: O((V+E) log V); space O(V+E).

Java implementation

```
import java.util.*;

class Dijkstra {
    static class Edge {
        int to, weight;
        Edge(int t, int w) { to=t; weight=w; }
    }
    static long[] shortest(List<List<Edge>> g, int source) {
        int n=g.size();
        long INF=Long.MAX_VALUE/4;
        long[] dist=new long[n];
        Arrays.fill(dist, INF);
        dist[source]=0;
        PriorityQueue<long[]> pq =
            new PriorityQueue<>(Comparator.comparingLong(a -> a[1]));
        pq.offer(new long[]{source,0});

        while (!pq.isEmpty()) {
            long[] cur=pq.poll();
            int u=(int)cur[0]; long d=cur[1];
            if (d != dist[u]) continue; // stale heap entry
            for (Edge e:g.get(u)) {
                long nd=d+e.weight;
                if (nd<dist[e.to]) {
                    dist[e.to]=nd;
                    pq.offer(new long[]{e.to,nd});
                }
            }
        }
        return dist;
    }
}
```

Dry run / example: Edges 0→1(4), 0→2(1), 2→1(2), 1→3(1), 2→3(5). From 0: dist[2]=1, dist[1]=3, dist[3]=4.

Important edge cases: Never use Dijkstra with negative edges. Use Bellman-Ford for negative edge weights (that is not a greedy algorithm). For undirected edges, add both directions.

6. Array, string and monotonic-stack greedy patterns

6.1 Jump Game I

Core idea: Determine whether the last index is reachable when each element is the maximum jump length from that index.

Greedy choice / rule: Track the farthest reachable index. If the current index is beyond it, fail; otherwise extend the reach.

Complexity: Time O(n); space O(1).

Java implementation

```
class JumpGame {
    static boolean canJump(int[] nums) {
        int farthest=0;
        for (int i=0;i<nums.length;i++) {
            if (i>farthest) return false;
            farthest=Math.max(farthest, i+nums[i]);
            if (farthest>=nums.length-1) return true;
        }
        return nums.length<=1;
    }
}
```

Dry run / example: nums [2,3,1,1,4]. From index 0 reach 2; index 1 extends reach to 4. Return true.

Important edge cases: Empty input handling may be defined differently by the platform. Watch integer overflow if jump values can be extremely large.

6.2 Jump Game II (minimum jumps)

Core idea: Find the minimum jumps to reach the final index; assume it is reachable unless problem says otherwise.

Greedy choice / rule: Treat the current jump range as a BFS-like layer. Scan the layer, compute the farthest next reach, then commit one jump when the layer ends.

Complexity: Time O(n); space O(1).

Java implementation

```
class JumpGameII {
    static int jump(int[] nums) {
        if (nums.length<=1) return 0;
        int jumps=0, currentEnd=0, farthest=0;
        for (int i=0;i<nums.length-1;i++) {
            farthest=Math.max(farthest, i+nums[i]);
            if (i==currentEnd) {
                jumps++;
                currentEnd=farthest;
                if (currentEnd<=i) return -1; // unreachable safeguard
            }
        }
        return jumps;
    }
}
```

Dry run / example: nums [2,3,1,1,4]. First range [0..0] extends to 2 (jump 1); next range [1..2] extends to 4 (jump 2). Answer = 2.

Important edge cases: If reachability is not guaranteed, return -1 when no progress is possible. This is a greedy layer-expansion pattern.

6.3 Gas Station

Core idea: Find a starting station to complete a circular route when gas[i] is gained and cost[i] is spent to reach the next station.

Greedy choice / rule: If total gas is less than total cost, impossible. Otherwise, when running tank becomes negative at i, no start from the current candidate through i can work; set candidate to i+1.

Complexity: Time O(n); space O(1).

Java implementation

```
class GasStation {
    static int canCompleteCircuit(int[] gas, int[] cost) {
        int total=0, tank=0, start=0;
        for (int i=0;i<gas.length;i++) {
            int diff=gas[i]-cost[i];
            total+=diff;
            tank+=diff;
            if (tank<0) {
                start=i+1;
                tank=0;
            }
        }
```

```
        return total>=0 ? (start==gas.length ? 0 : start) : -1;
    }
}
```

Dry run / example: gas [1,2,3,4,5], cost [3,4,5,1,2]. Total surplus is non-negative; start at index 3 succeeds.

Important edge cases: Arrays must have equal length. If multiple starts work, this returns one valid start.

6.4 Partition Labels

Core idea: Split a string into as many parts as possible so each character appears in at most one part.

Greedy choice / rule: Record the last index of each character. While scanning, extend the current partition end to the farthest last occurrence seen; close when i reaches that end.

Complexity: Time O(n); space O(1) for fixed alphabet.

Java implementation

```
import java.util.*;

class PartitionLabels {
    static List<Integer> partitionLabels(String s) {
        int[] last=new int[26];
        for (int i=0;i<s.length();i++) last[s.charAt(i)-'a']=i;
        List<Integer> ans=new ArrayList<>();
        int start=0, end=0;
        for (int i=0;i<s.length();i++) {
            end=Math.max(end,last[s.charAt(i)-'a']);
            if (i==end) {
                ans.add(end-start+1);
                start=i+1;
            }
        }
        return ans;
    }
}
```

Dry run / example: String ababcbacadefegdehijhklij gives partition sizes [9,7,8].

Important edge cases: This code assumes lowercase English letters. Use a map for general Unicode or arbitrary character sets.

6.5 Remove K Digits

Core idea: Remove exactly k digits from a non-negative integer string to make the smallest possible number.

Greedy choice / rule: Maintain a monotonic increasing stack. When the incoming digit is smaller than the stack top, remove larger previous digits while removals remain.

Complexity: Time O(n); space O(n).

Java implementation

```
class RemoveKDigits {
    static String removeKdigits(String num, int k) {
        StringBuilder st=new StringBuilder();
        for (char c:num.toCharArray()) {
            while (k>0 && st.length()>0 &&
                   st.charAt(st.length()-1)>c) {
                st.deleteCharAt(st.length()-1);
                k--;
            }
            st.append(c);
        }
        while (k>0 && st.length()>0) {
            st.deleteCharAt(st.length()-1);
            k--;
        }
        int i=0;
        while (i<st.length() && st.charAt(i)=='0') i++;
        String ans=st.substring(i);
        return ans.isEmpty() ? "0" : ans;
    }
}
```

Dry run / example: num='1432219', k=3. Remove 4, 3, and one 2 according to the monotonic rule; result='1219'.

Important edge cases: Handle k equal to number length, leading zeros, and already nondecreasing digits.

6.6 Candy distribution

Core idea: Give each child at least one candy; a child with a higher rating than an adjacent child must get more candies. Minimize total candies.

Greedy choice / rule: Left-to-right pass enforces the rising-from-left constraint. Right-to-left pass enforces the rising-from-right constraint by taking the maximum of both requirements.

Complexity: Time O(n); space O(n).

Java implementation

```
class Candy {
    static int candy(int[] ratings) {
        int n=ratings.length;
        if (n==0) return 0;
        int[] c=new int[n];
        java.util.Arrays.fill(c,1);
        for (int i=1;i<n;i++)
            if (ratings[i]>ratings[i-1]) c[i]=c[i-1]+1;
        for (int i=n-2;i>=0;i--)
            if (ratings[i]>ratings[i+1])
                c[i]=Math.max(c[i],c[i+1]+1);
        int sum=0;
        for (int x:c) sum+=x;
        return sum;
    }
}
```

Dry run / example: Ratings [1,0,2]. Candies become [2,1,2], total = 5.

Important edge cases: Use long for the sum if n can be very large.

6.7 Wiggle Subsequence

Core idea: Find the length of the longest subsequence whose consecutive differences alternate positive and negative.

Greedy choice / rule: Track the best length ending with an upward move and downward move; a new rise improves up, a new fall improves down.

Complexity: Time O(n); space O(1).

Java implementation

```
class WiggleSubsequence {
    static int wiggleMaxLength(int[] nums) {
        if (nums.length==0) return 0;
        int up=1, down=1;
        for (int i=1;i<nums.length;i++) {
            if (nums[i]>nums[i-1]) up=down+1;
            else if (nums[i]<nums[i-1]) down=up+1;
        }
        return Math.max(up,down);
    }
}
```

Dry run / example: nums [1,7,4,9,2,5]. Alternating rises/falls allow all six elements; answer = 6.

Important edge cases: Equal adjacent values do not change either state. This computes subsequence, not contiguous subarray.

6.8 Queue reconstruction by height

Core idea: Reconstruct people ordered by height and the number k of people in front who have height at least theirs.

Greedy choice / rule: Sort by height descending, and for equal heights by k ascending. Insert each person at index k in a list.

Complexity: Time O(n²) with ArrayList insertion; space O(n).

Java implementation

```
import java.util.*;

class QueueReconstruction {
```

```
    static int[][] reconstructQueue(int[][] people) {
        Arrays.sort(people, (a,b) -> {
            if (a[0]!=b[0]) return Integer.compare(b[0],a[0]);
            return Integer.compare(a[1],b[1]);
        });
        List<int[]> q=new ArrayList<>();
        for (int[] p:people) q.add(p[1],p);
        return q.toArray(new int[q.size()][]);
    }
}
```

Dry run / example: people [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]. Insert tallest first; shorter people do not change the count of taller/equal people already ahead.

Important edge cases: Index k must be valid at insertion. This standard formulation uses height descending and k ascending for ties.

6.9 Lemonade Change

Core idea: Serve customers paying with 5, 10, or 20 when lemonade costs 5; determine whether correct change can always be returned.

Greedy choice / rule: Track counts of 5 and 10 bills. For a 20, prefer one 10 + one 5; otherwise use three 5s.

Complexity: Time O(n); space O(1).

Java implementation

```
class LemonadeChange {
    static boolean lemonadeChange(int[] bills) {
        int five=0, ten=0;
        for (int b:bills) {
            if (b==5) five++;
            else if (b==10) {
                if (five==0) return false;
                five--; ten++;
            } else {
                if (ten>0 && five>0) { ten--; five--; }
                else if (five>=3) five-=3;
                else return false;
            }
        }
        return true;
    }
}
```

Dry run / example: bills [5,5,5,10,20]. For 20, give 10+5, preserving two 5s. Return true.

Important edge cases: The preference for 10+5 preserves more 5-dollar bills for future 10-dollar customers.

7. Additional high-value greedy interview patterns

7.1 Boats to Save People

Core idea: Carry at most two people per boat under a weight limit; minimize boats.

Greedy choice / rule: Sort weights. Pair the lightest with the heaviest if they fit; otherwise send the heaviest alone.

Complexity: Time O(n log n); space O(1) beyond sorting.

Java implementation

```
import java.util.*;

class BoatsToSavePeople {
    static int numRescueBoats(int[] people, int limit) {
        Arrays.sort(people);
        int l=0, r=people.length-1, boats=0;
        while (l<=r) {
            if (people[l]+people[r]<=limit) l++;
            r--;
            boats++;
        }
        return boats;
    }
}
```

Dry run / example: weights [3,2,2,1], limit 3. Pair 1+2, pair 2 alone, pair 3 alone. Boats = 3.

Important edge cases: Assumes at most two people per boat. Use long for sum if weights may overflow int.

7.2 Bag of Tokens

Core idea: Maximize score by playing a token face-up (spend power, gain score) or face-down (spend score, gain power).

Greedy choice / rule: Sort tokens. If possible, play the cheapest face-up; otherwise, if score allows and a token remains, play the most expensive face-down to gain power.

Complexity: Time O(n log n); space O(1) beyond sorting.

Java implementation

```
import java.util.*;

class BagOfTokens {
    static int bagOfTokensScore(int[] tokens, int power) {
        Arrays.sort(tokens);
        int l=0, r=tokens.length-1, score=0, best=0;
        while (l<=r) {
            if (power>=tokens[l]) {
                power-=tokens[l++];
                score++;
                best=Math.max(best,score);
            } else if (score>0 && l<r) {
                power+=tokens[r--];
                score--;
            } else break;
        }
        return best;
    }
}
```

Dry run / example: tokens [100,200,300,400], power 200. Play 100 face-up (score 1), 400 face-down (power 500), then 200 and 300 face-up. Best score = 2.

Important edge cases: Track best score, not just final score, because a later face-down move can reduce it.

7.3 Task Scheduler (cooldown)

Core idea: Schedule tasks with identical task types separated by at least n idle intervals; minimize total intervals.

Greedy choice / rule: Count frequencies. The most frequent task determines the frame length; fill gaps with other tasks.

Complexity: Time O(N + alphabet) or O(N log alphabet) depending on implementation; space O(alphabet).

Java implementation

```
import java.util.*;

class TaskScheduler {
    static int leastInterval(char[] tasks, int n) {
        int[] f=new int[26];
        for (char c:tasks) f[c-'A']++;
        Arrays.sort(f);
        int max=f[25], same=1;
        for (int i=24;i>=0 && f[i]==max;i--) same++;
        int frame=(max-1)*(n+1)+same;
        return Math.max(tasks.length,frame);
    }
}
```

Dry run / example: Tasks A A A B B B, cooldown 2. A _ _ A _ _ A requires 8 slots; B fills some gaps, final minimum = 8.

Important edge cases: Formula assumes one task per interval and identical task types have cooldown n. Verify the exact cooldown definition in the prompt.

7.4 Reorganize String

Core idea: Rearrange characters so no equal adjacent characters appear.

Greedy choice / rule: Always place the currently most frequent remaining character next; use a max-heap and temporarily hold the previous character for one step.

Complexity: Time O(n log alphabet); space O(alphabet).

Java implementation

```
import java.util.*;

class ReorganizeString {
    static String reorganizeString(String s) {
        int[] f=new int[26];
        for (char c:s.toCharArray()) f[c-'a']++;
        PriorityQueue<int[]> pq =
            new PriorityQueue<>((a,b)->Integer.compare(b[1],a[1]));
        for (int i=0;i<26;i++) if (f[i]>0) pq.offer(new int[]{i,f[i]});

        StringBuilder ans=new StringBuilder();
        int[] prev=null;
        while (!pq.isEmpty()) {
            int[] cur=pq.poll();
            ans.append((char)('a'+cur[0]));
            cur[1]--;
            if (prev!=null && prev[1]>0) pq.offer(prev);
            prev=cur;
        }
        return ans.length()==s.length() ? ans.toString() : "";
    }
}
```

Dry run / example: s='aab'. Output can be 'aba'. For 'aaab', no valid arrangement exists, so return empty string.

Important edge cases: Feasibility condition for alphabet size m: maxFrequency <= ceil(n/2). This implementation assumes lowercase English letters.

7.5 Furthest Building You Can Reach

Core idea: Climb height differences using bricks or ladders; maximize reachable building index.

Greedy choice / rule: Use ladders for the largest climbs. Put every positive climb in a min-heap; whenever heap size exceeds ladder count, pay the smallest climb with bricks.

Complexity: Time O(n log L); space O(L), where L is number of ladders (heap can temporarily hold L+1).

Java implementation

```
import java.util.*;

class FurthestBuilding {
    static int furthestBuilding(int[] h, int bricks, int ladders) {
        PriorityQueue<Integer> climbs=new PriorityQueue<>();
        for (int i=0;i<h.length-1;i++) {
            int d=h[i+1]-h[i];
            if (d>0) climbs.offer(d);
            if (climbs.size()>ladders) bricks-=climbs.poll();
            if (bricks<0) return i;
        }
        return h.length-1;
    }
}
```

Dry run / example: Use ladders on the largest climbs; if there are more climbs than ladders, bricks pay for the smallest. Return the last reachable index.

Important edge cases: Use long if height differences or brick counts can exceed int.

7.6 Maximum events you can attend

Core idea: Attend the maximum number of events, each available from start day through end day, at most one per day.

Greedy choice / rule: Sort by start day. For each day, add events that have started to a min-heap by end day; discard expired events and attend the one ending earliest.

Complexity: Time O(n log n + D log n), where D is number of processed days; event-driven implementations can reduce idle-day scanning.

Java implementation

```java
import java.util.*;

class MaxEvents {
    static int maxEvents(int[][] events) {
        Arrays.sort(events, Comparator.comparingInt(a -> a[0]));
        PriorityQueue<Integer> ends=new PriorityQueue<>();
        int i=0, day=0, attended=0, n=events.length;
        while (i<n || !ends.isEmpty()) {
            if (ends.isEmpty()) day=Math.max(day,events[i][0]);
            while (i<n && events[i][0]<=day) ends.offer(events[i++][1]);
            while (!ends.isEmpty() && ends.peek()<day) ends.poll();
            if (!ends.isEmpty()) {
                ends.poll();
                attended++;
                day++;
            }
        }
        return attended;
    }
}
```

Dry run / example: Events [1,2], [2,3], [3,4]. Attend one on each of days 1,2,3. Answer = 3.

Important edge cases: An event ending on day d is still attendable on day d, so remove only end < day.

7.7 Maximum number of consecutive ones after flipping at most k zeros

Core idea: Find the longest subarray containing at most k zeros.

Greedy choice / rule: Sliding window: expand right; while zero count exceeds k, move left until valid.

Complexity: Time O(n); space O(1).

Java implementation

```java
class MaxConsecutiveOnesIII {
    static int longestOnes(int[] nums, int k) {
        int left=0, zeros=0, best=0;
        for (int right=0;right<nums.length;right++) {
            if (nums[right]==0) zeros++;
            while (zeros>k) {
                if (nums[left]==0) zeros--;
                left++;
            }
            best=Math.max(best,right-left+1);
        }
        return best;
    }
}
```

Dry run / example: nums [1,1,1,0,0,0,1,1,1,1,0], k=2. Best window length = 6.

Important edge cases: This is a greedy two-pointer/sliding-window pattern; it relies on the window validity being restored by moving left.

8. Important problems that are NOT solved by a simple greedy rule

Interview questions often use the word 'minimum' or 'maximum' but still require DP, graph algorithms, or a proof beyond a simple local choice.

• **0/1 Knapsack:** a single value/weight sort is not generally optimal; use DP.

• **Coin Change for arbitrary denominations:** greedy can fail (counterexample in Section 1); use DP unless the coin system is proven canonical.

• **Weighted interval scheduling:** choosing earliest finish alone is not optimal; sort by end + DP with binary search.

• **Minimum jumps with arbitrary costs / path costs:** may require DP or shortest path, not Jump Game's reachability greedy.

• **Shortest path with negative edges:** Dijkstra is invalid; Bellman-Ford is a standard alternative.

• **Set cover:** repeatedly choosing the set covering the most uncovered elements is an approximation heuristic, not always exact.

• **General bin packing:** first-fit-decreasing is an approximation, not an exact universal greedy solution.

Interview rule: Never say 'greedy works because it chooses the best'. State the exact rule and justify why a better global solution cannot be lost.

9. Greedy proof templates

Exchange argument

1. Let G be the first greedy choice and O be an optimal solution. 2. If O already contains G, continue with the remaining subproblem. 3. Otherwise replace O's conflicting choice with G. 4. Show feasibility is preserved and objective does not worsen. 5. Therefore an optimal solution exists that starts with G.

Stays-ahead argument

After every step i, show the greedy solution is at least as far ahead / has no larger cost / has selected no fewer compatible items than any competing solution. This is useful for interval scheduling and reachability.

Cut property for MST

For any cut of a weighted undirected graph, a minimum-weight edge crossing that cut is safe for some MST. Kruskal and Prim repeatedly use this safety principle.

Dijkstra invariant

When the minimum tentative-distance vertex is extracted from the priority queue, its distance is final only because all edge weights are non-negative. A negative edge can later produce a shorter route, breaking the invariant.

10. Interview implementation checklist

• Sort comparator: use Integer.compare(a, b), not a[0] - b[0], to avoid overflow.

• Intervals: decide whether touching endpoints count as overlap; encode the rule consistently.

• PriorityQueue: Java is a min-heap by default. Use Collections.reverseOrder() for a max-heap.

• Stale heap entries: in Dijkstra, skip a popped state if its distance no longer equals dist[node].

• DSU: use path compression + union by size/rank.

• Numeric safety: use long for path distances, accumulated costs, rope sums, and large totals.

• Empty input / one element / all equal values / impossible case: test each explicitly.

• State the invariant before coding: what does the current variable or heap represent?

• Prove the greedy decision; if you cannot, test counterexamples and consider DP.

11. Practice bank: implement these yourself

Intervals and scheduling

■ Activity Selection / Maximum Meetings in One Room

■ Non-overlapping Intervals

■ Minimum Number of Arrows to Burst Balloons

■ Minimum Number of Meeting Rooms

■ Minimum Platforms

- Video Stitching
- Minimum Taps to Water a Garden
- Employee Free Time
- Maximum Events That Can Be Attended
- Meeting Rooms II

Knapsack, jobs and resources

- Fractional Knapsack
- Job Sequencing with Deadlines
- Assign Cookies
- Maximum Units on a Truck
- IPO / Maximize Capital
- Minimum Refueling Stops
- Boats to Save People
- Bag of Tokens
- Maximum Performance of a Team (heap + sorting)
- Course Schedule III (deadline + max-heap)

Heap and frequency

- Huffman Coding
- Connect Ropes with Minimum Cost
- Task Scheduler
- Reorganize String
- Furthest Building You Can Reach
- Top K Frequent Elements (heap selection)
- Minimum Cost to Hire K Workers (ratio sort + heap)
- Smallest Range Covering Elements from K Lists

Arrays and strings

- Jump Game I
- Jump Game II
- Gas Station
- Partition Labels
- Remove K Digits
- Candy
- Wiggle Subsequence
- Queue Reconstruction by Height
- Lemonade Change
- Maximum Length of Pair Chain
- Longest Subarray of Ones After Deleting One Element

■ Valid Parenthesis String (greedy low/high bounds)

Graphs

■ Kruskal Minimum Spanning Tree

■ Prim Minimum Spanning Tree

■ Dijkstra Shortest Path

■ Minimum Cost to Connect All Points (MST)

■ Network Delay Time (Dijkstra)

■ Path With Minimum Effort (priority queue / minimax path)

12. Complexity quick reference

Algorithm / pattern	Time	Extra space (typical)
Activity selection	O(n log n)	O(1) after sort
Meeting rooms (two sorted arrays)	O(n log n)	O(n)
Fractional knapsack	O(n log n)	O(1) after sort
Job sequencing (simple slots)	O(n log n + nD)	O(D)
Huffman / connect ropes	O(n log n)	O(n)
Kruskal MST	O(E log E)	O(V)
Prim MST (heap)	O((V+E) log V)	O(V+E)
Dijkstra (heap)	O((V+E) log V)	O(V+E)
Jump Game / Gas Station	O(n)	O(1)
Remove K Digits	O(n)	O(n)
Candy	O(n)	O(n)
Task Scheduler	O(n + alphabet)	O(alphabet)

Final preparation advice: For every pattern, first learn the greedy rule and its proof intuition, then implement it without looking, dry-run a boundary case, and solve the practice bank. Knowing code without knowing why the greedy choice is safe is not enough for interviews.